Below is a **MASTER mapping table** that shows:

✅ **Microservice design patterns**
✅ which **deployment type fits best**
✅ and which **DevOps/Cloud stack** is normally used for each pattern (**Jenkins, GitHub, GitLab, Docker, Kubernetes, AWS, Azure, GCP**)

✅ Microservice Design Pattern → Suitable Deployment Type → Tool/Platform Mapping (MASTER TABLE)

Legend (Deployment Types):

- **VM/Server-based** = Deploy on Linux servers / EC2 / VMSS / Compute Engine
- **Container-based** = Docker container runtime
- **Kubernetes-based** = Pods/Deployments/StatefulSets/DaemonSets
- **Serverless-based** = Functions & managed services
- **Hybrid** = combination (common in enterprise)

#	Microservice Design Pattern	What it Solves	Best Deployment Type	Jenkins Pipeline Fit	GitHub / GitLab Fit	Docker Fit	Kubernetes Fit	AWS	Azure	GCP
1	**API Gateway**	Single entry point	Kubernetes / VM	✅ CD pipeline	✅	✅	✅ Ingress + Gateway	API Gateway	API Mgmt	API Gateway
2	**Backend For Frontend (BFF)**	Separate API for UI apps	Kubernetes / Container	✅	✅	✅	✅	ECS/EKS	AKS	GKE
3	**Service Discovery**	Find service instances dynamically	Kubernetes (Native)	✅	✅	✅	✅ (Core Feature)	Cloud Map/EKS	AKS + DNS	GKE + DNS

4	**Client-side Discovery**	client finds service	VM/Container	✓	✓	✓	☐ less common	EC2 + Eureka	VMSS + Eureka	GCE + Eureka
5	**Server-side Discovery**	router/load balancer finds service	Kubernetes / Cloud LB	✓	✓	✓	✓	ALB/NLB	Azure LB	Cloud LB
6	**Circuit Breaker**	Prevent cascading failure	Any	✓	✓	✓	✓	Resilience libs	Resilience libs	Resilience libs
7	**Bulkhead**	Isolate failures	Kubernetes preferred	✓	✓	✓	✓	EKS	AKS	GKE
8	**Retry + Timeout**	Handle transient errors	Any	✓	✓	✓	✓	SDK + Env	SDK + Env	SDK + Env
9	**Rate Limiting**	Control traffic	Gateway / Ingress	✓	✓	✓	✓	WAF/ API GW	API Mgmt	API GW
10	**Load Balancer Pattern**	Distribute requests	Kubernetes / Cloud LB	✓	✓	✓	✓	ELB	Azure LB	Cloud LB
11	**Database per Service**	Microservices autonomy	Kubernetes/ Hybrid	✓	✓	✓	✓	RDS/Dynamo DB	SQL/Cosmos DB	CloudSQL/Firestore

12	**Shared Database (Anti-pattern mostly)**	Fast integration but tight coupling	VM/Hybrid	✓	✓	✓	✓	RDS	SQL DB	CloudSQL
13	**Saga (Choreography)**	Distributed transactions via events	Kubernetes	✓	✓	✓	✓	SNS/SQS	Service Bus	Pub/Sub
14	**Saga (Orchestration)**	Transaction controlled by orchestrator	Kubernetes / Container	✓	✓	✓	✓	Step Functions	Durable Functions	Workflows
15	**Event-Driven Architecture**	Async integration	Kubernetes/ Serverless	✓	✓	✓	✓	Event Bridge	Event Grid	Eventarc
16	**CQRS**	Separate read/write models	Kubernetes/ Hybrid	✓	✓	✓	✓	DynamoDB + OpenSearch	Cosmos + Search	Firestore + BigQuery
17	**Event Sourcing**	Store events not state	Kubernetes	✓	✓	✓	✓	Kinesis	Event Hubs	Pub/Sub

18	**Strangler Fig Pattern**	Gradual migration monolith →microservices	Hybrid	✓	✓	✓	✓	ECS/EKS	AKS	GKE
19	**Sidecar Pattern**	Proxy/logging/security per service	Kubernetes	✓	✓	✓	✓(Native)	App Mesh	Service Mesh	Anthos Mesh/Istio
20	**Ambassador Pattern**	Local proxy per service	Kubernetes	✓	✓	✓	✓	Envoy	Envoy	Envoy
21	**Adapter Pattern**	Connect legacy to new services	Any	✓	✓	✓	✓	Lambda/ECS	Functions/AKS	Cloud Run/GKE
22	**Anti-Corruption Layer (ACL)**	Protect domain from legacy	Any	✓	✓	✓	✓	ECS/EKS	AKS	GKE
23	**Externalized Configuration**	env configs centralized	Kubernetes	✓	✓	✓	✓ ConfigMap/Secrets	SSM	App Config	Secret Manager
24	**Centralized Logging**	Observability	Kubernetes best	✓	✓	✓	✓	Cloud Watch	Monitor	Cloud Logging

25	**Distributed Tracing**	Track request chain	Kubernetes best	✅	✅	✅	✅	X-Ray	App Insights	Cloud Trace
26	**Health Check Pattern**	Liveness /readiness	Kubernetes	✅	✅	✅	✅	ALB health	Probe	Probe
27	**Blue-Green Deployment**	zero downtime release	Kubernetes / VM	✅Best	✅	✅	✅	CodeDeploy/ EKS	DevOps/AKS	Cloud Deploy
28	**Canary Deployment**	safer gradual rollout	Kubernetes	✅	✅	✅	✅	App Mesh	AKS	Cloud Deploy
29	**Rolling Deployment**	gradual replace pods	Kubernetes	✅	✅	✅	✅	EKS	AKS	GKE
30	**Shadow Deployment**	duplicate traffic to new version	Kubernetes	✅	✅	✅	✅	App Mesh	Traffic Manager	Traffic Director
31	**Service Mesh Pattern**	mTLS, retries, routing	Kubernetes	✅	✅	✅	✅ Strong	Istio/AppMesh	Istio/ OSM	Istio/Anthos
32	**API Composition**	aggregate multiple APIs	Kubernetes	✅	✅	✅	✅	Lambda/ECS	Functions	Cloud Run

33	**Aggregator Pattern**	gather results from services	Kubernetes	✓	✓	✓	✓	ECS/Lambda	AKS/Func	GKE/Run
34	**Command Pattern**	isolate commands	Any	✓	✓	✓	✓	ECS	AKS	GKE
35	**Compensating Transaction**	undo operation if failure	Kubernetes	✓	✓	✓	✓	Step Functions	Durable Func	Workflows
36	**Message Broker Pattern**	async communication	Kubernetes/ Hybrid	✓	✓	✓	✓	SQS/SNS/MSK	Service Bus	Pub/Sub
37	**Idempotency Pattern**	avoid duplicates	Any	✓	✓	✓	✓	DynamoDB	CosmosDB	Firestore
38	**Security Token Pattern (JWT/OAuth)**	auth standardization	Gateway/K8s	✓	✓	✓	✓	Cognito	AAD B2C	Identity Platform
39	**Secrets Management Pattern**	protect creds	Kubernetes	✓	✓	✓	✓	Secrets Manager	Key Vault	Secret Manager
40	**Autoscaling Pattern**	scale services automatically	Kubernetes	✓	✓	✓	✓ HPA/VPA	ASG/EKS	AKS autoscale	GKE autoscale

---

✅ Quick Conclusion (Very Important)

In real-time enterprise DevOps:

- **Most microservice patterns deploy best on Kubernetes**
- Jenkins/GitHub/GitLab are mainly for **CI/CD**
- Docker is for **packaging**
- AWS/Azure/GCP are for **runtime + managed services**
- Kubernetes is for **orchestration**

If you want, I can also give you **a 2nd table**:

✅ Pattern → **Exact Jenkins pipeline stages** (Checkout → Build → Test → Docker build → Push → Deploy → Verify)
with **sample Jenkinsfile blocks** for each deployment model (VM / Docker / Kubernetes / Cloud).